

1. Exercício 1 – Criando um Novo Endpoint GET

Crie um novo endpoint GET em `ContactController` que permita buscar contatos pelo nome.

Requisitos:

- O método deve receber o nome como um parâmetro de URL (`/api/contacts/search?name=João`).
- O método deve retornar uma lista de contatos que correspondam ao nome fornecido.
- Caso nenhum contato seja encontrado, retorne uma lista vazia.

Dicas:

- Você pode precisar modificar a interface `ContactRepository` para adicionar um método de busca personalizada.

Saída esperada (JSON):

```
[
  {
    "id": 1,
    "nome": "João Silva",
    "telefone": "9999-9999",
    "email": "joao@email.com"
  }
]
```

Print do primeiro exercício:

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `http://localhost:8080/api/contacts/search?name=João`
- Send Button:** A purple button labeled "Send".
- Query Parameters:**
 - ☒ **name**: João
 - ☐ **parameter**: value
- Status:** 200
- Size:** 89 Bytes
- Time:** 470 ms
- Response:** A JSON array containing one object:

```
[
  {
    "nome": "João da Silva",
    "telefone": "9999-9999",
    "email": "joaodasilva@email.com",
    "id": 1
  }
]
```

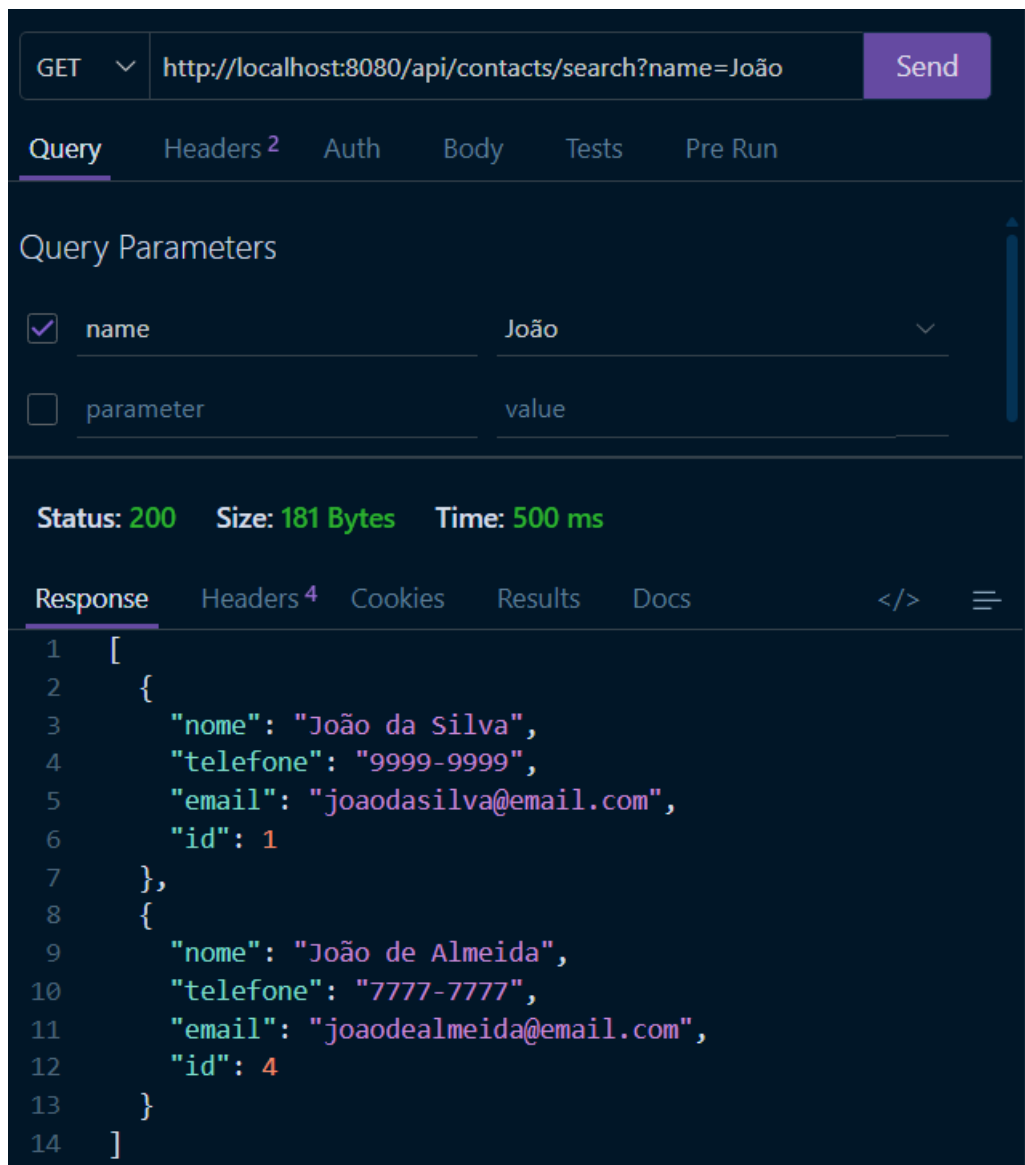
Teste com contato inexistente:

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `http://localhost:8080/api/contacts/search?name=Luiz`
- Send Button:** A purple button labeled "Send".
- Query Parameters:**
 - ☒ **name**: Luiz
 - ☐ **parameter**: value
- Status:** 200
- Size:** 2 Bytes
- Time:** 1.73 s
- Response:** An empty JSON array:

```
[]
```

Teste com mais um contato “João” adicionado:



Exercício 2 - Implementando um Método PATCH

Adicione um novo método PATCH à API, permitindo que o usuário atualize apenas um ou mais campos de um contato, sem precisar enviar todos os dados.

Requisitos:

- O método deve permitir alterar apenas os campos enviados na requisição.
- Se o campo não for enviado, o valor original deve ser mantido.
- Retorne o contato atualizado após a alteração.
- Caso o ID fornecido não exista, retorne um erro 404.

Exemplo de chamada:

PATCH /api/contacts/1

```
{  
  "email": "novoemail@email.com"  
}
```

Saída esperada (JSON):

```
{  
  "id": 1,  
  "nome": "João Silva",  
  "telefone": "9999-9999",  
  "email": "novoemail@email.com"  
}
```

Print do segundo exercício:

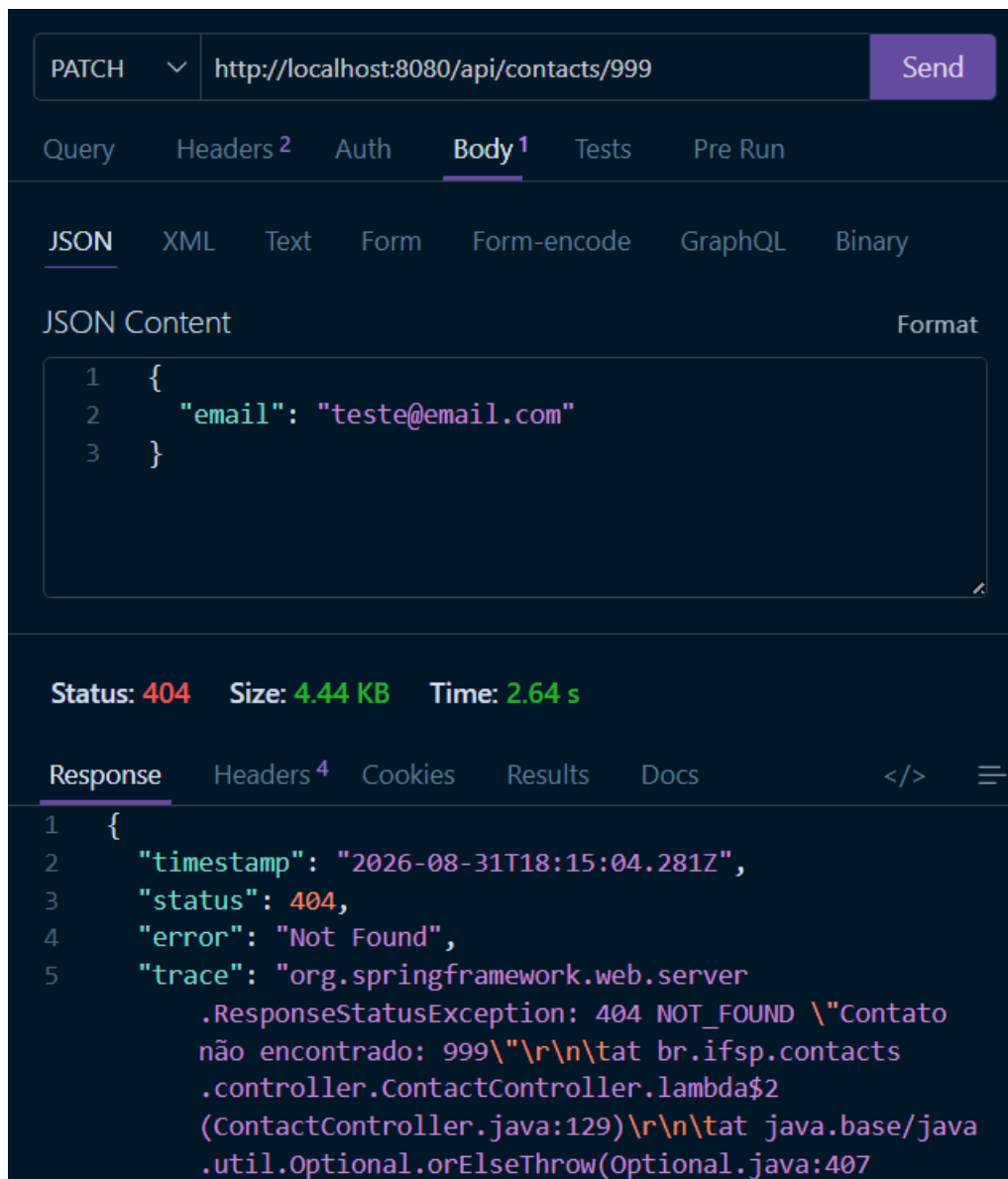
The screenshot shows a REST client interface with the following details:

- Method:** PATCH
- URL:** http://localhost:8080/api/contacts/1
- Body:** JSON Content

```
1 {  
2   "email": "novoemail@email.com"  
3 }
```
- Status:** 200
- Size:** 87 Bytes
- Time:** 4.18 s
- Response:** JSON Content

```
1 {  
2   "nome": "João de Almeida",  
3   "telefone": "7777-7777",  
4   "email": "novoemail@email.com",  
5   "id": 1  
6 }
```

Teste com ID inexistente e retorno 404 Not Found:



Exercício 3 - Comparando REST e SOAP

Agora que você já conhece APIs REST, faça uma pesquisa sobre APIs SOAP e responda às perguntas abaixo com suas próprias palavras.

Questões:

1. Qual a principal diferença entre REST e SOAP?

REST é uma forma de criar APIs usando os recursos do HTTP, como GET, POST, PUT e DELETE. SOAP é um protocolo que utiliza mensagens estruturadas, geralmente em XML, para a comunicação entre sistemas.

De forma simples: REST é mais simples e flexível, enquanto SOAP é mais padronizado e possui mais regras.

2. Em quais cenários SOAP ainda é amplamente utilizado?

SOAP ainda é bastante usado em sistemas bancários, governamentais, seguradoras e grandes empresas, principalmente quando são necessárias muita segurança, confiabilidade e regras bem definidas. Também aparece em sistemas antigos que ainda precisam funcionar e se comunicar com outros sistemas.

3. Quais são as vantagens e desvantagens de usar REST ao invés de SOAP?

Vantagens do REST: É mais simples de desenvolver e utilizar; geralmente utiliza JSON, que é mais fácil de ler; funciona muito bem com sites e aplicativos; é mais leve e flexível.

Desvantagens: Alguns recursos mais específicos, como segurança e transações complexas, precisam ser implementados separadamente; pode não ser a melhor escolha para sistemas que exigem padrões empresariais muito específicos.

4. O que é WS-Security e como ele se compara à segurança em APIs REST?

WS-Security é um conjunto de padrões de segurança usado em SOAP. Ele permite proteger as mensagens usando recursos como assinaturas digitais, criptografia e tokens.

No REST, a segurança normalmente é feita com HTTPS, autenticação, autorização e tokens. Portanto, REST não é necessariamente menos seguro; ele apenas utiliza mecanismos diferentes e, geralmente, mais simples.

5. Explique o modelo de maturidade de Richardson e em que nível SOAP se encaixa.

O modelo de Richardson mostra o quanto uma API segue os princípios do REST. Ele possui quatro níveis:

- Nível 0: HTTP é usado apenas como transporte.
- Nível 1: utiliza diferentes endereços para representar recursos.
- Nível 2: utiliza corretamente GET, POST, PUT e DELETE.

- Nível 3: além dos anteriores, a própria resposta indica ao cliente quais ações ele pode realizar.

O SOAP não se encaixa diretamente no modelo, pois não é REST. Porém, em uma comparação, um serviço SOAP tradicional pode ser considerado próximo do Nível 0, já que normalmente usa o HTTP apenas para transportar suas mensagens.